

Lecture 2 - Part 2: Introduction to the Cloud and Apache Spark

Note: Don't run all cells in the notebook by default. Read every cell carefully before deciding to run it.

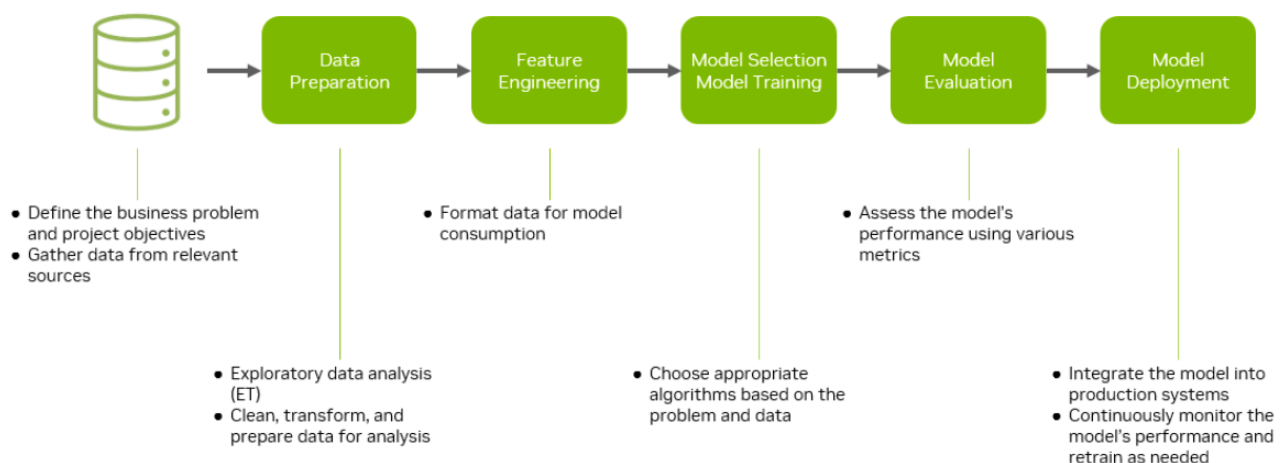
Overview

- Introduction to the Cloud & Context
- Basics of Apache Spark
- Run Apache Spark on the Cloud

Why would you go with the Cloud?



Connect the Dots!



Context

Last lecture, we talked about the machine learning modeling process. Now, let's switch gears and discuss our technical infrastructure that can be deployed later to the cloud. You will need Spark installed on your machine in order to run the code snippets in this lecture.

In this class, we will use **Apache Spark** for data preparation, cleaning, and feature engineering. But why Spark?

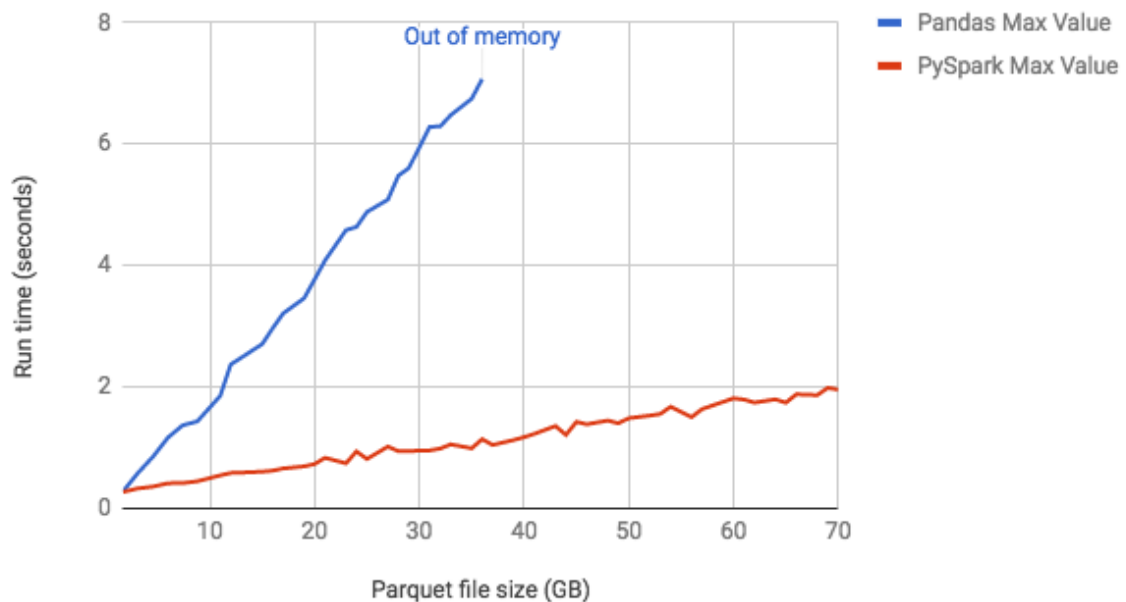
Why Spark?

- Developed in 2009 at UC Berkeley AMPLab, then open sourced in 2010,
- Spark is the next revolution of the popular Hadoop MapReduce framework.
- Gartner, Advanced Analytics and Data Science (2014) "Organizations that are looking at big data challenges – including collection, ETL, storage, exploration and analytics – should consider Spark for its in-memory performance and the breadth of its model. It supports advanced analytics solutions on Hadoop clusters, including the iterative model required for machine learning and graph analysis."

Pandas vs PySpark (Spark developed in Python)

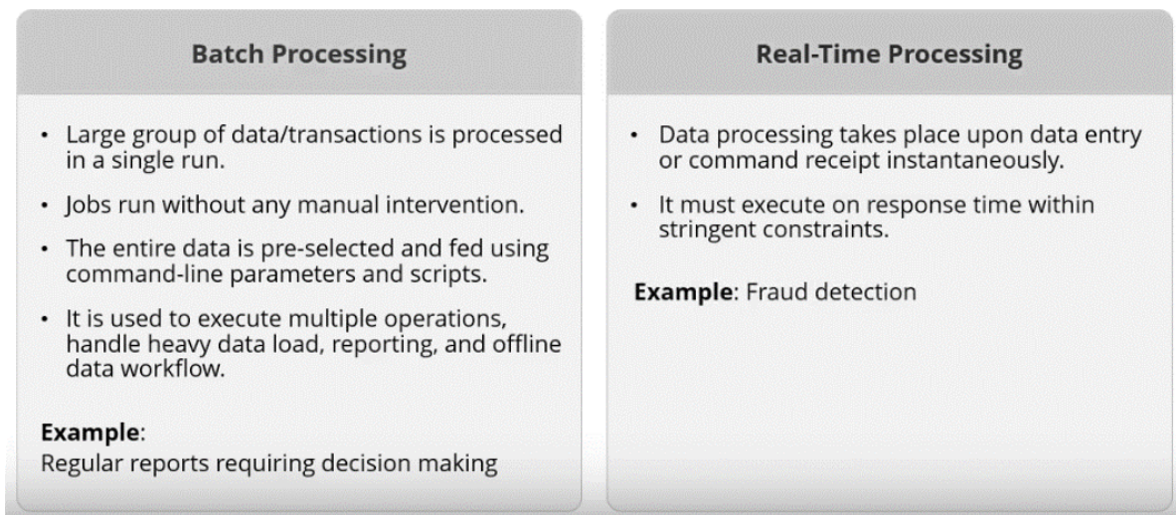
In a research done by Databricks, an industrial leader in the domain of big data storage and processing, PySpark shows superior performance compared to traditional implementations.

Pandas VS PySpark: max value

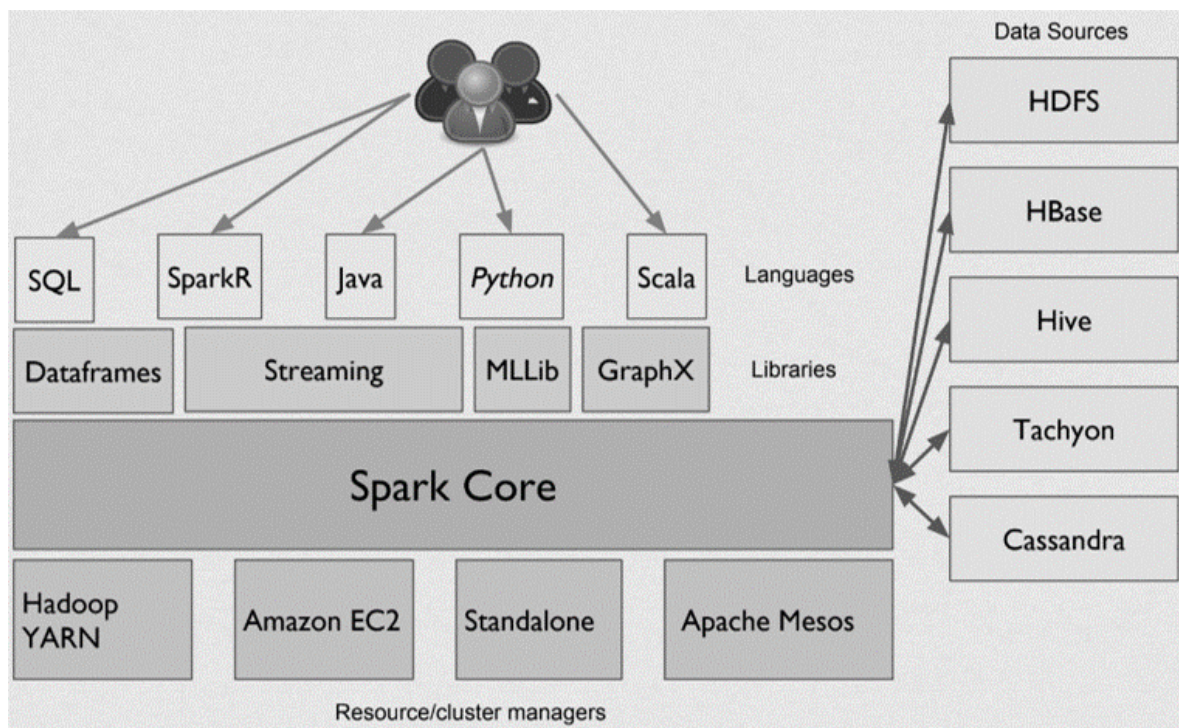


What is Spark?

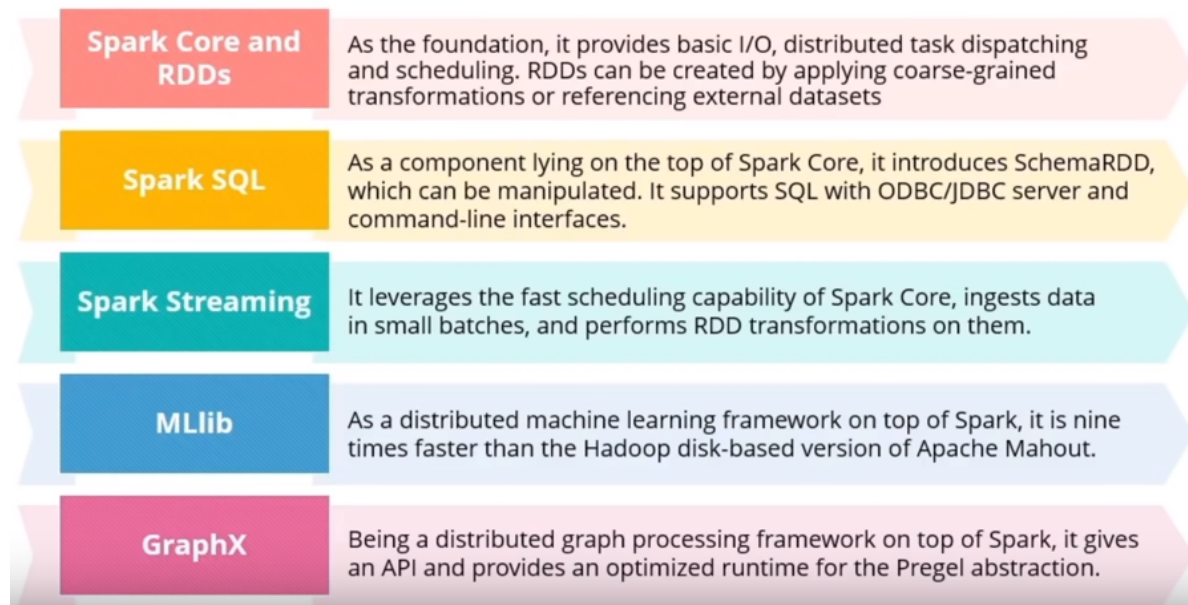
- Apache Spark is a fast and general-purpose cluster computing system for large scale data processing.
- Spark was originally written in Scala, which allows concise function syntax and interactive use.
- Apache Spark provides High-level APIs in Java, Scala, Python (PySpark) and R.
- Apache Spark combines two different modes of processing:
 - **Batch-based Processing** which can be provided via Apache Hadoop MapReduce
 - **Real-time Processing** which can be provided via Apache Storm.



Spark Ecosystem



Spark Componentenets



Spark Core

Spark Core is the general execution engine for the Spark platform that other functionalities are built on top of it. Spark has several advantages:

- **Speed:** runs programs up to 100x faster than Hadoop MapReduce in memory, or 10x faster on disk
- **Ease of Use:** Write applications quickly in Java, Scala, Python, R
- **Generality:** Combine SQL, streaming, and complex analytics
- **Runs Everywhere:** Spark runs on Hadoop, Mesos, standalone, or in the cloud. It can access diverse data sources including HDFS, Cassandra, HBase, and S3

```
In [1]: # General Test for Your Jupyter Kernel Version
# The below code snippet will generated an error if you are using outdated python version
import sys

print(sys.version)
print(sys.executable)

assert sys.version_info >= (3, 11)

3.14.5 (tags/v3.14.5:5607950, May 10 2026, 10:43:50) [MSC v.1944 64 bit (AMD64)]
C:\Python314\python.exe
```

```
In [1]: # if you installed Spark on windows,
# you may need findspark and need to initialize it prior to being able to use pyspark

!pip install findspark

Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: findspark in C:\Users\mfarag\AppData\Roaming\Python\Python314\site-packages (2.0.1)

[notice] A new release of pip is available: 26.1.1 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

```
In [2]: #Run the python library command after you install spark to your machine according to the
#!pip install pyspark

Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: pyspark in C:\Users\mfarag\AppData\Roaming\Python\Python314\site-packages (4.2.0)
Collecting py4j<0.10.9.10,>=0.10.9.7 (from pyspark)
  Downloading py4j-0.10.9.9-py2.py3-none-any.whl.metadata (1.3 kB)
  Downloading py4j-0.10.9.9-py2.py3-none-any.whl (203 kB)
Installing collected packages: py4j
Successfully installed py4j-0.10.9.9

[notice] A new release of pip is available: 26.1.1 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

```
In [3]: # Uncomment the following lines if you are using Windows!
#import findspark
#findspark.init()
#findspark.find()
# The above lines are used mostly in Windows only and you don't need them on other platform
import pyspark
from pyspark.sql import SparkSession
spark = SparkSession.builder.master("local[*]").appName("SparkTest").getOrCreate()
```


Download Data Files Remotely

```
In [5]: !pip install wget
```

```
Defaulting to user installation because normal site-packages is not writeable
Collecting wget
  Downloading wget-3.2.zip (10 kB)
  Installing build dependencies: started
  Installing build dependencies: finished with status 'done'
  Getting requirements to build wheel: started
  Getting requirements to build wheel: finished with status 'done'
  Preparing metadata (pyproject.toml): started
  Preparing metadata (pyproject.toml): finished with status 'done'
Building wheels for collected packages: wget
  Building wheel for wget (pyproject.toml): started
  Building wheel for wget (pyproject.toml): finished with status 'done'
  Created wheel for wget: filename=wget-3.2-py3-none-any.whl size=9694 sha256=fa007abd9c400c5499de24c0693b7c55721f4be5a8a793a666523e331cc10110
  Stored in directory: c:\users\mfarag\appdata\local\pip\cache\wheels\9c\b9\25\66d2377ed05ab1424fa6b361f1088fc5ae065f96efad7202dc
Successfully built wget
Installing collected packages: wget
Successfully installed wget-3.2

[notice] A new release of pip is available: 26.1.1 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

```
In [7]: # Downloading and preprocessing KDD Train Data
!python -m wget https://www.andrew.cmu.edu/user/mfarag/763/KDDTrain+.txt
```

Saved under KDDTrain+ (1).txt

Cloud Consideration

Your data need to be moved to a special storage server if you are running Spark on the Cloud. This special storage is called HDFS. The following command is used to move your data from your local storage to the special storage server.

```
In [ ]: # Uncomment and Execute this line if you are running your notebook on the Cloud
#!hadoop fs -put KDDTrain+.txt /
```

Spark Dataframes

Inspired by pandas DataFrames in structure, format, and a few specific operations, Spark DataFrames are like distributed in-memory tables with named columns and schemas, where each column has a specific data type: integer, string, array, map, real, date, timestamp, etc. To a human's eye, a Spark DataFrame is like a table

When data are visualized as a structured table, it's not only easy to digest but also easy to work with when it comes to common operations you might want to execute on rows and columns.

Also, DataFrames are immutable and Spark keeps a lineage of all transformations. You can add or change the names and data types of the columns, creating new DataFrames while the previous versions are preserved. A named column in a DataFrame and its associated Spark data type can be declared in the schema.

Let's examine the generic and structured data types available in Spark before we use them to define a schema. Then we'll illustrate how to create a DataFrame with a schema.

Spark's Basic Data Types

Spark supports basic internal data types. These data types can be declared in your Spark application or defined in your schema

Data type	Value assigned in Python	API to instantiate
ByteType	int	DataTypes.ByteType
ShortType	int	DataTypes.ShortType
IntegerType	int	DataTypes.IntegerType
LongType	int	DataTypes.LongType
FloatType	float	DataTypes.FloatType
DoubleType	float	DataTypes.DoubleType
StringType	str	DataTypes.StringType
BooleanType	bool	DataTypes.BooleanType
DecimalType	decimal.Decimal	DecimalType

```
In [8]: # Load data from csv to a dataframe on a local machine.
# header=False means the first row is not a header
# sep=',' means the column are seperated using ','
df = spark.read.csv('KDDTrain+.txt', header=False, sep=",")
# on the Cloud, the files will have to be at the root level. So, the cloud version is:
#df = spark.read.csv('/KDDTrain+.txt', header=False, sep=",")

df.show(5, vertical=True)
```

```
-RECORD 0-----
```

```
_c0 | 0
_c1 | tcp
_c2 | ftp_data
_c3 | SF
_c4 | 491
_c5 | 0
_c6 | 0
_c7 | 0
_c8 | 0
_c9 | 0
_c10 | 0
_c11 | 0
_c12 | 0
_c13 | 0
_c14 | 0
_c15 | 0
_c16 | 0
_c17 | 0
_c18 | 0
_c19 | 0
```



```
_c20 | 0
_c21 | 0
_c22 | 2
_c23 | 2
_c24 | 0.00
_c25 | 0.00
_c26 | 0.00
_c27 | 0.00
_c28 | 1.00
_c29 | 0.00
_c30 | 0.00
_c31 | 150
_c32 | 25
_c33 | 0.17
_c34 | 0.03
_c35 | 0.17
_c36 | 0.00
_c37 | 0.00
_c38 | 0.00
_c39 | 0.05
_c40 | 0.00
_c41 | normal
_c42 | 20
-RECORD 1-----
_c0 | 0
_c1 | udp
_c2 | other
_c3 | SF
_c4 | 146
_c5 | 0
_c6 | 0
_c7 | 0
_c8 | 0
_c9 | 0
_c10 | 0
_c11 | 0
_c12 | 0
_c13 | 0
_c14 | 0
_c15 | 0
_c16 | 0
_c17 | 0
_c18 | 0
_c19 | 0
_c20 | 0
_c21 | 0
_c22 | 13
_c23 | 1
_c24 | 0.00
_c25 | 0.00
_c26 | 0.00
_c27 | 0.00
_c28 | 0.08
_c29 | 0.15
_c30 | 0.00
_c31 | 255
_c32 | 1
_c33 | 0.00
_c34 | 0.60
_c35 | 0.88
_c36 | 0.00
_c37 | 0.00
_c38 | 0.00
_c39 | 0.00
_c40 | 0.00
_c41 | normal
```

```
_c42 | 15
-RECORD 2-----
_c0 | 0
_c1 | tcp
_c2 | private
_c3 | S0
_c4 | 0
_c5 | 0
_c6 | 0
_c7 | 0
_c8 | 0
_c9 | 0
_c10 | 0
_c11 | 0
_c12 | 0
_c13 | 0
_c14 | 0
_c15 | 0
_c16 | 0
_c17 | 0
_c18 | 0
_c19 | 0
_c20 | 0
_c21 | 0
_c22 | 123
_c23 | 6
_c24 | 1.00
_c25 | 1.00
_c26 | 0.00
_c27 | 0.00
_c28 | 0.05
_c29 | 0.07
_c30 | 0.00
_c31 | 255
_c32 | 26
_c33 | 0.10
_c34 | 0.05
_c35 | 0.00
_c36 | 0.00
_c37 | 1.00
_c38 | 1.00
_c39 | 0.00
_c40 | 0.00
_c41 | neptune
_c42 | 19
-RECORD 3-----
_c0 | 0
_c1 | tcp
_c2 | http
_c3 | SF
_c4 | 232
_c5 | 8153
_c6 | 0
_c7 | 0
_c8 | 0
_c9 | 0
_c10 | 0
_c11 | 1
_c12 | 0
_c13 | 0
_c14 | 0
_c15 | 0
_c16 | 0
_c17 | 0
_c18 | 0
_c19 | 0
```

```
_c20 | 0
_c21 | 0
_c22 | 5
_c23 | 5
_c24 | 0.20
_c25 | 0.20
_c26 | 0.00
_c27 | 0.00
_c28 | 1.00
_c29 | 0.00
_c30 | 0.00
_c31 | 30
_c32 | 255
_c33 | 1.00
_c34 | 0.00
_c35 | 0.03
_c36 | 0.04
_c37 | 0.03
_c38 | 0.01
_c39 | 0.00
_c40 | 0.01
_c41 | normal
_c42 | 21
-RECORD 4-----
_c0 | 0
_c1 | tcp
_c2 | http
_c3 | SF
_c4 | 199
_c5 | 420
_c6 | 0
_c7 | 0
_c8 | 0
_c9 | 0
_c10 | 0
_c11 | 1
_c12 | 0
_c13 | 0
_c14 | 0
_c15 | 0
_c16 | 0
_c17 | 0
_c18 | 0
_c19 | 0
_c20 | 0
_c21 | 0
_c22 | 30
_c23 | 32
_c24 | 0.00
_c25 | 0.00
_c26 | 0.00
_c27 | 0.00
_c28 | 1.00
_c29 | 0.00
_c30 | 0.09
_c31 | 255
_c32 | 255
_c33 | 1.00
_c34 | 0.00
_c35 | 0.00
_c36 | 0.00
_c37 | 0.00
_c38 | 0.00
_c39 | 0.00
_c40 | 0.00
_c41 | normal
```

_c42 | 21
only showing top 5 rows

Avoiding Auto-assigned Column Names: Read CSV and Specify the Column Names

```
In [9]: col_names = ["duration", "protocol_type", "service", "flag", "src_bytes",
    "dst_bytes", "land", "wrong_fragment", "urgent", "hot", "num_failed_logins",
    "logged_in", "num_compromised", "root_shell", "su_attempted", "num_root",
    "num_file_creations", "num_shells", "num_access_files", "num_outbound_cmds",
    "is_host_login", "is_guest_login", "count", "srv_count", "serror_rate",
    "srv_serror_rate", "rerror_rate", "srv_rerror_rate", "same_srv_rate",
    "diff_srv_rate", "srv_diff_host_rate", "dst_host_count", "dst_host_srv_count",
    "dst_host_same_srv_rate", "dst_host_diff_srv_rate", "dst_host_same_src_port_rate",
    "dst_host_srv_diff_host_rate", "dst_host_serror_rate", "dst_host_srv_serror_rate",
    "dst_host_rerror_rate", "dst_host_srv_rerror_rate", "classes", "difficulty_level"]

df = spark.read.csv("KDDTrain+.txt", header=False, inferSchema= True).toDF(*col_names)

# on the Cloud, the files will have to be at the root level. So, the cloud version is:
# df = spark.read.csv("/KDDTrain+.txt", header=False, inferSchema= True).toDF(*col_names)

df.show(1, vertical=True)
```

-RECORD 0-----

duration	0
protocol_type	tcp
service	ftp_data
flag	SF
src_bytes	491
dst_bytes	0
land	0
wrong_fragment	0
urgent	0
hot	0
num_failed_logins	0
logged_in	0
num_compromised	0
root_shell	0
su_attempted	0
num_root	0
num_file_creations	0
num_shells	0
num_access_files	0
num_outbound_cmds	0
is_host_login	0
is_guest_login	0
count	2
srv_count	2
serror_rate	0.0
srv_serror_rate	0.0
rerror_rate	0.0
srv_rerror_rate	0.0
same_srv_rate	1.0
diff_srv_rate	0.0
srv_diff_host_rate	0.0
dst_host_count	150
dst_host_srv_count	25
dst_host_same_srv_rate	0.17
dst_host_diff_srv_rate	0.03
dst_host_same_src_port_rate	0.17
dst_host_srv_diff_host_rate	0.0
dst_host_serror_rate	0.0
dst_host_srv_serror_rate	0.0
dst_host_rerror_rate	0.05
dst_host_srv_rerror_rate	0.0

```
classes | normal
difficulty_level | 20
only showing top 1 row
```

More ways to Display Dataframes

1. `df.take(5)` will return a list of five Row objects.
2. `df.collect()` will get all of the data from the entire DataFrame. Be really careful when using it, because if you have a large data set, you can easily crash the driver node.
3. `df.show()` is the most commonly used method to view a dataframe. There are a few parameters we can pass to this method, like the number of rows and truncate. For example, `df.show(5, False)` or `df.show(5, truncate=False)` will show the entire data without any truncation.
4. `df.limit(5)` will **return a new DataFrame** by taking the first n rows. As spark is distributed in nature, there is no guarantee that `df.limit()` will give you the same results each time.

Display Schema Information for Your Dataframe

```
In [10]: df.printSchema() # or df.dtypes
```

```
root
|-- duration: integer (nullable = true)
|-- protocol_type: string (nullable = true)
|-- service: string (nullable = true)
|-- flag: string (nullable = true)
|-- src_bytes: integer (nullable = true)
|-- dst_bytes: integer (nullable = true)
|-- land: integer (nullable = true)
|-- wrong_fragment: integer (nullable = true)
|-- urgent: integer (nullable = true)
|-- hot: integer (nullable = true)
|-- num_failed_logins: integer (nullable = true)
|-- logged_in: integer (nullable = true)
|-- num_compromised: integer (nullable = true)
|-- root_shell: integer (nullable = true)
|-- su_attempted: integer (nullable = true)
|-- num_root: integer (nullable = true)
|-- num_file_creations: integer (nullable = true)
|-- num_shells: integer (nullable = true)
|-- num_access_files: integer (nullable = true)
|-- num_outbound_cmds: integer (nullable = true)
|-- is_host_login: integer (nullable = true)
|-- is_guest_login: integer (nullable = true)
|-- count: integer (nullable = true)
|-- srv_count: integer (nullable = true)
|-- serror_rate: double (nullable = true)
|-- srv_serror_rate: double (nullable = true)
|-- rerror_rate: double (nullable = true)
|-- srv_rerror_rate: double (nullable = true)
|-- same_srv_rate: double (nullable = true)
|-- diff_srv_rate: double (nullable = true)
|-- srv_diff_host_rate: double (nullable = true)
|-- dst_host_count: integer (nullable = true)
|-- dst_host_srv_count: integer (nullable = true)
|-- dst_host_same_srv_rate: double (nullable = true)
|-- dst_host_diff_srv_rate: double (nullable = true)
|-- dst_host_same_src_port_rate: double (nullable = true)
|-- dst_host_srv_diff_host_rate: double (nullable = true)
|-- dst_host_serror_rate: double (nullable = true)
|-- dst_host_srv_serror_rate: double (nullable = true)
|-- dst_host_rerror_rate: double (nullable = true)
|-- dst_host_srv_rerror_rate: double (nullable = true)
|-- classes: string (nullable = true)
|-- difficulty_level: integer (nullable = true)
```

Print Column Names in Your Dataframe

```
In [11]: print(df.columns)
```

```
['duration', 'protocol_type', 'service', 'flag', 'src_bytes', 'dst_bytes', 'land', 'wrong_fragment', 'urgent', 'hot', 'num_failed_logins', 'logged_in', 'num_compromised', 'root_shell', 'su_attempted', 'num_root', 'num_file_creations', 'num_shells', 'num_access_files', 'num_outbound_cmds', 'is_host_login', 'is_guest_login', 'count', 'srv_count', 'serror_rate', 'srv_serror_rate', 'rerror_rate', 'srv_rerror_rate', 'same_srv_rate', 'diff_srv_rate', 'srv_diff_host_rate', 'dst_host_count', 'dst_host_srv_count', 'dst_host_same_srv_rate', 'dst_host_diff_srv_rate', 'dst_host_same_src_port_rate', 'dst_host_srv_diff_host_rate']
```

```
t_rate', 'dst_host_serror_rate', 'dst_host_srv_serror_rate', 'dst_host_rerror_rate', 'ds  
t_host_srv_rerror_rate', 'classes', 'difficulty_level']
```

Print Total Number of Your Records in Your Dataframe

```
In [12]: print(df.count())
```

```
125973
```

Print Sample Record from Your Dataframe

```
In [13]: df.show(1, vertical=True)
```

```
-RECORD 0-----  
duration                | 0  
protocol_type           | tcp  
service                 | ftp_data  
flag                    | SF  
src_bytes                | 491  
dst_bytes               | 0  
land                    | 0  
wrong_fragment          | 0  
urgent                  | 0  
hot                     | 0  
num_failed_logins       | 0  
logged_in               | 0  
num_compromised         | 0  
root_shell              | 0  
su_attempted            | 0  
num_root                | 0  
num_file_creations      | 0  
num_shells               | 0  
num_access_files        | 0  
num_outbound_cmds       | 0  
is_host_login           | 0  
is_guest_login          | 0  
count                   | 2  
srv_count               | 2  
serror_rate             | 0.0  
srv_serror_rate         | 0.0  
rerror_rate             | 0.0  
srv_rerror_rate         | 0.0  
same_srv_rate           | 1.0  
diff_srv_rate           | 0.0  
srv_diff_host_rate      | 0.0  
dst_host_count          | 150  
dst_host_srv_count      | 25  
dst_host_same_srv_rate  | 0.17  
dst_host_diff_srv_rate  | 0.03  
dst_host_same_src_port_rate | 0.17  
dst_host_srv_diff_host_rate | 0.0  
dst_host_serror_rate    | 0.0  
dst_host_srv_serror_rate | 0.0  
dst_host_rerror_rate    | 0.05  
dst_host_srv_rerror_rate | 0.0  
classes                 | normal  
difficulty_level        | 20  
only showing top 1 row
```

Lab: Run Spark on the Cloud

You will not be able to run the notebooks on the cloud until you redeem the GCP coupons in week-3 (after course enrollment is finalized). For now, keep track of the steps to repeat them after you redeem your GCP coupons. Don't use a free trial on GCP for your account. Coupons can't be redeemed on accounts with free trial on them.

Create Clusters (e.g. Hadoop Clusters)

- A cluster is group of machines, servers, or nodes. It helps providing the sum of the computational power offered by all incorporated machines. It's difficult to build a local machine with 64GB RAM and 20TB of Storage but that is not difficult when you are running on the cloud.
- You may start by navigating to "Managed Apache Spark" and click on the Clusters section

Cluster Configuration

Make sure to follow the cluster creation guide posted on Canvas (and enable the required APIs). **You must redeem your GCP coupons prior to creating the cluster. You will receive the information about GCP coupons in week-3.**

Running Cluster

Once you click on the create button, Google Cloud will work on creating your own cluster and if it's successful, you will see your cluster running.

Filter Search clusters, press Enter								
☐	Name ↓	Status	Region	Zone	Total worker nodes	Scheduled deletion	Cloud Storage staging bucket	Created
☐	cluster-d712	✔ Running	us-central1	us-central1-f	2	Off	dataproc-staging-us-central1-934687243262-r97j0sab	Jan 26, 2022, 11:43:28 PM

Connect to Your Cluster

From the Web Interfaces, open Jupyter. Upload your Notebook to **GCS** folder and run the cells.